

Land-Use Classification of Satellite Images (EuroSAT)

MLP vs CNN vs Transfer Learning (ResNet18)

Deep Learning — Midterm Project
Yerlan Utepbergenov, Group BDA-2401

Abstract

We classify Sentinel-2 satellite images from the EuroSAT RGB dataset (27,000 images, 64×64×3, 10 land-use classes) with three models trained on the same fixed 70/15/15 train/validation/test split: a multilayer perceptron (MLP), a custom convolutional neural network (CNN), and an ImageNet-pretrained ResNet18 fine-tuned with a new 10-class output layer. The MLP reaches only **61.6%** test accuracy despite 6.4M parameters, because flattening destroys spatial structure. The CNN reaches **90.3%** with three times fewer parameters (2.2M). Transfer learning gives the best result, **97.1%**, after only 3 epochs. An optimizer comparison shows that Adam converges fastest (85.7% validation accuracy after 5 epochs vs 58.9% for plain SGD), and a dropout experiment shows that dropout reduces the train–validation gap from 5.1% to 1.9%. Feature visualization shows that the first convolutional layer detects edges, colours and simple textures.

1. Introduction

Problem. Given a 64×64 RGB satellite image, predict one of 10 land-use classes: AnnualCrop, Forest, HerbaceousVegetation, Highway, Industrial, Pasture, PermanentCrop, Residential, River, SeaLake.

Motivation. Automatic land-use mapping helps agriculture monitoring, urban planning and environmental protection. Classes differ mainly in texture and spatial patterns (field stripes, street grids, river shapes), which are hard to describe with hand-made rules but can be learned by deep networks, especially CNNs.

Approach. (1) MLP baseline, (2) custom CNN, (3) comparison of four optimizers, (4) dropout regularization experiment, (5) transfer learning with ResNet18, (6) evaluation with accuracy, loss curves, confusion matrices, parameter counts, training time and feature visualization. Implemented in PyTorch; all seeds fixed (42).

2. Data and Preprocessing

Dataset: EuroSAT RGB (Helber et al., 2019), downloaded automatically with `torchvision.datasets.EuroSAT`. 27,000 labelled images, 10 classes, 64×64 pixels, 3 channels (RGB).



Figure 1. Two example images from each class.

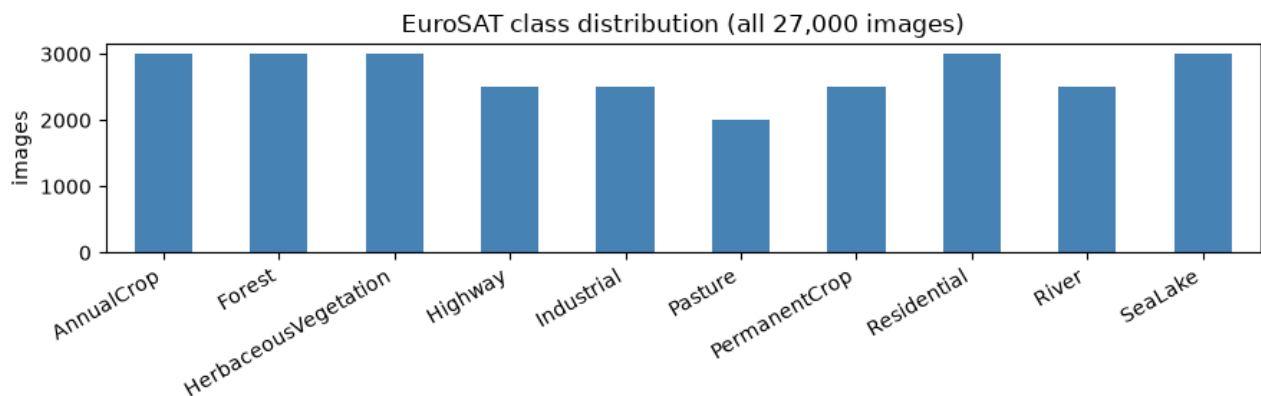


Figure 2. Class distribution (all 27,000 images).

The dataset is **mildly imbalanced**: 3,000 images for most classes, 2,500 for Highway, Industrial, PermanentCrop and River, and 2,000 for Pasture. Accuracy is still a fair metric; confusion matrices show per-class errors.

Split	Images	Share	Purpose
Train	18,900	70%	learn the weights
Validation	4,050	15%	monitor training, compare settings
Test	4,050	15%	final evaluation only

Preprocessing

- **ToTensor**: pixels scaled to $[0, 1]$.
- **Normalize**: per-channel mean (0.344, 0.380, 0.408) and std (0.203, 0.137, 0.116), computed on the **training set only** to avoid data leakage.
- **Augmentation (train only)**: random horizontal and vertical flips — a satellite image has no fixed orientation, so flipped images keep their class.

3. Methods

Training setup (all models): Cross-Entropy loss, batch size 128, accuracy as metric, same data split and seed.

3.1 MLP baseline

Layer	Details
Input	Flatten $3 \times 64 \times 64 = 12,288$
Hidden 1	Linear 12,288 to 512, ReLU, Dropout 0.2
Hidden 2	Linear 512 to 256, ReLU, Dropout 0.2
Output	Linear 256 to 10 (logits)

Adam optimizer, learning rate 0.001, 10 epochs. Trainable parameters: **6,425,866**.

3.2 Custom CNN

Block	Layers	Output
Input	–	$3 \times 64 \times 64$
Block 1	Conv 3 to 32 ($k=3, s=1, p=1$), ReLU, MaxPool 2	$32 \times 32 \times 32$
Block 2	Conv 32 to 64 ($k=3, s=1, p=1$), ReLU, MaxPool 2	$64 \times 16 \times 16$
Block 3	Conv 64 to 128 ($k=3, s=1, p=1$), ReLU, MaxPool 2	$128 \times 8 \times 8$
Classifier	Flatten 8,192, Linear 256, ReLU, Dropout 0.5, Linear 10	10

- **Kernel 3×3 :** small and cheap; stacked 3×3 layers grow the receptive field.
- **Filters 32, 64, 128:** the image shrinks after each pooling, so the number of channels grows to keep capacity for more complex patterns.
- **Stride 1, padding 1:** the filter visits every position and the size is kept ("same" padding); downsampling is done by 2×2 max pooling.

Output-size calculation ($O = (W - K + 2P) / S + 1$) for Conv1: $W = 64, K = 3, P = 1, S = 1$, so $O = (64 - 3 + 2) / 1 + 1 = 64$, giving a $32 \times 64 \times 64$ feature map. MaxPool 2×2 then gives $(64 - 2) / 2 + 1 = 32$.

Parameters (conv: $(K \cdot K \cdot C_{in} + 1) \cdot C_{out}$): Conv1 = 896, Conv2 = 18,496, Conv3 = 73,856, FC1 = 2,097,408, FC2 = 2,570. Total trainable parameters: **2,193,226**. Training: Adam, lr 0.001, 10 epochs.

3.3 Transfer learning: ResNet18

Transfer learning reuses a network trained on a large dataset (ImageNet, 1.2M images) as a starting point. Its early layers already detect general features such as edges, colours and textures, which are also useful for satellite images, so the model learns faster and needs less data.

- Loaded ResNet18 with pretrained ImageNet weights (IMAGENET1K_V1).
- Replaced the final layer (512 to 1000) with a new layer (512 to 10).
- Fine-tuned all layers with Adam, small learning rate 0.0001, 3 epochs. ResNet uses residual (skip) connections, $y = F(x) + x$, which help gradients flow.
- Trainable parameters: **11,181,642**.

4. Optimization and Training Analysis

4.1 Optimizer comparison

The same CNN was trained with four optimizers using the same data split and the same initial weights, for 5 epochs each.

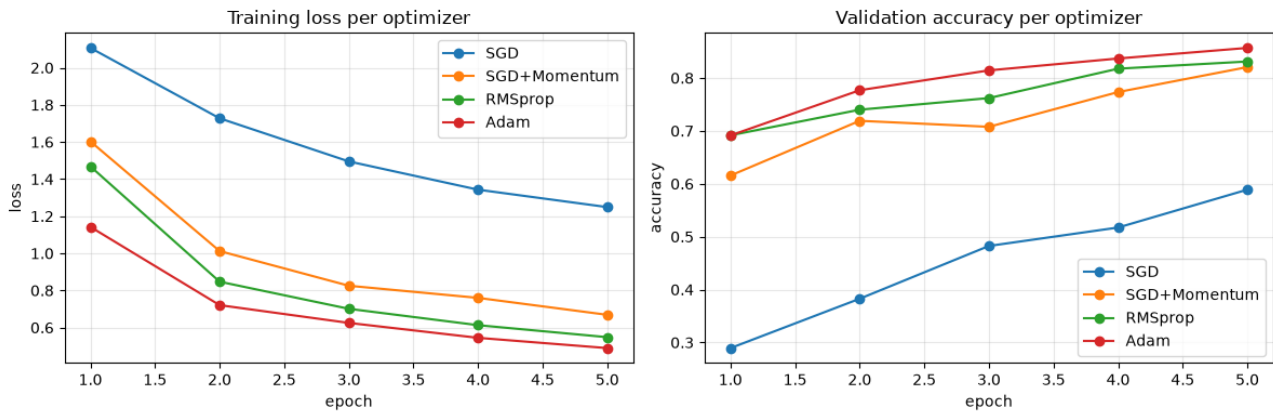


Figure 3. Training loss and validation accuracy per optimizer.

Optimizer	lr	Final train loss	Val. accuracy	Time
SGD	0.01	1.249	58.9%	70 s
SGD + Momentum (0.9)	0.01	0.669	82.0%	71 s
RMSprop	0.001	0.548	83.1%	70 s
Adam	0.001	0.489	85.7%	71 s

- **Adam** converges fastest: it combines momentum with a per-parameter adaptive step size.
- **Momentum** and **RMSprop** are much faster than plain SGD; momentum keeps speed in consistent directions, RMSprop adapts the step for each weight.
- **Plain SGD** learns slowly with a fixed small step (58.9% after 5 epochs).
- **Training time** is the same for all (about 70 s): the optimizer changes how many epochs are needed, not the cost per epoch.

4.2 Regularization: Dropout

Dropout randomly switches off neurons during training (here $p = 0.5$), so the network cannot rely on single neurons and learns more robust features. We compare the CNN with and without dropout, trained **without augmentation** so overfitting is visible.

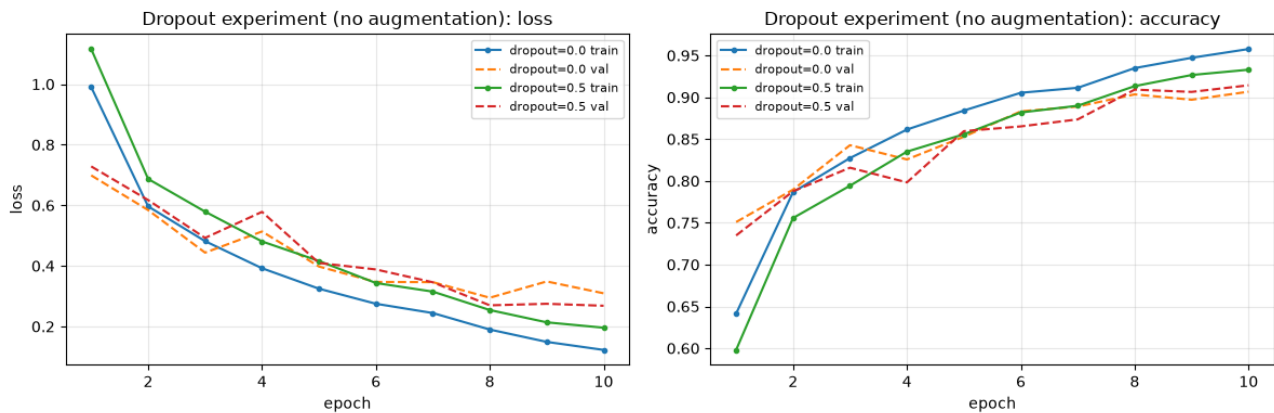


Figure 4. Loss and accuracy with and without dropout.

Model	Train acc.	Val. acc.	Gap	Final val. loss
No dropout	95.8%	90.7%	5.1%	0.309 (rising)
Dropout 0.5	93.3%	91.4%	1.9%	0.268

Without dropout the training loss keeps falling while the validation loss starts to rise after epoch 8 — a sign of **overfitting**. Dropout makes training slightly harder (lower train accuracy) but improves **generalization**: higher validation accuracy, lower validation loss and a much smaller train–validation gap.

5. Experiments and Results

5.1 Model comparison (test set)

Model	Test accuracy	Parameters	Epochs	Train time	Time / epoch
MLP	61.6%	6,425,866	10	77 s	8 s
CNN	90.3%	2,193,226	10	143 s	14 s
ResNet18 (TL)	97.1%	11,181,642	3	74 s	25 s

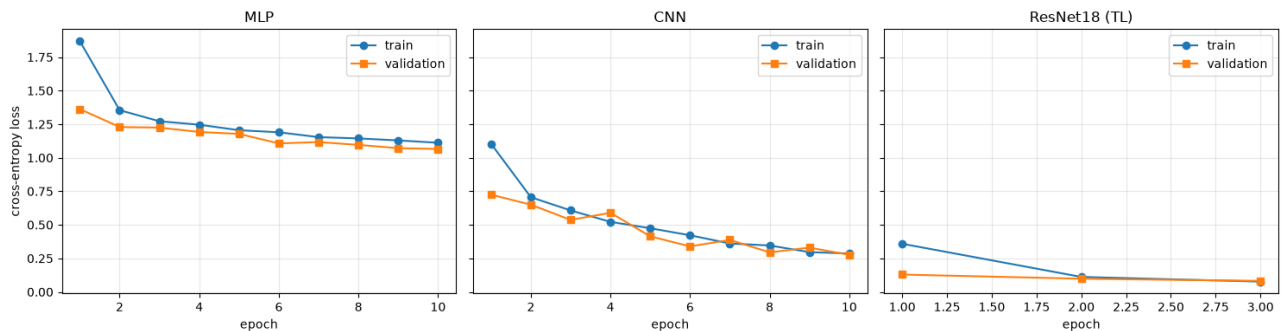


Figure 5. Training and validation loss curves for all three models.

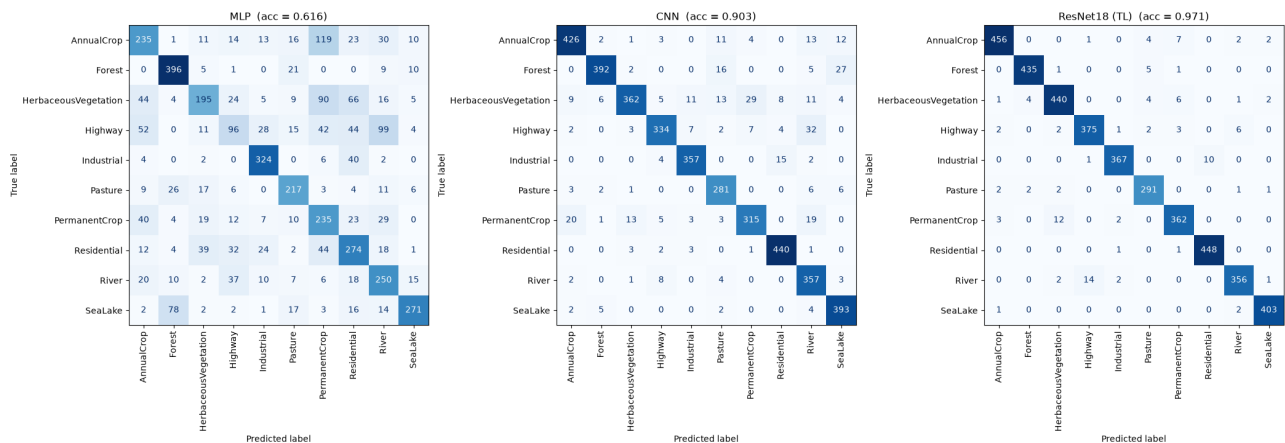


Figure 6. Confusion matrices on the test set (rows: true class, columns: predicted).

- **MLP**: many errors, e.g. AnnualCrop to PermanentCrop (119), Highway to River (99), SeaLake to Forest (78) — without spatial features it relies mostly on colour.
- **CNN**: remaining errors between similar classes: Highway and River (thin lines), HerbaceousVegetation and PermanentCrop, Forest and SeaLake (dark uniform textures).
- **ResNet18**: almost diagonal; small errors such as River to Highway (14) and Industrial to Residential (10).

5.2 Feature visualization

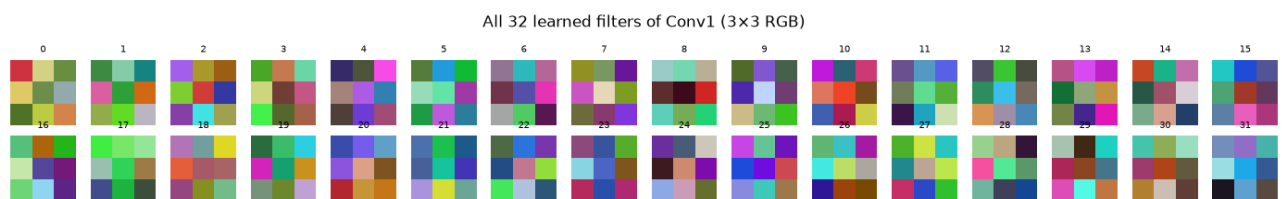


Figure 7. All 32 learned 3x3 filters of the first convolutional layer.

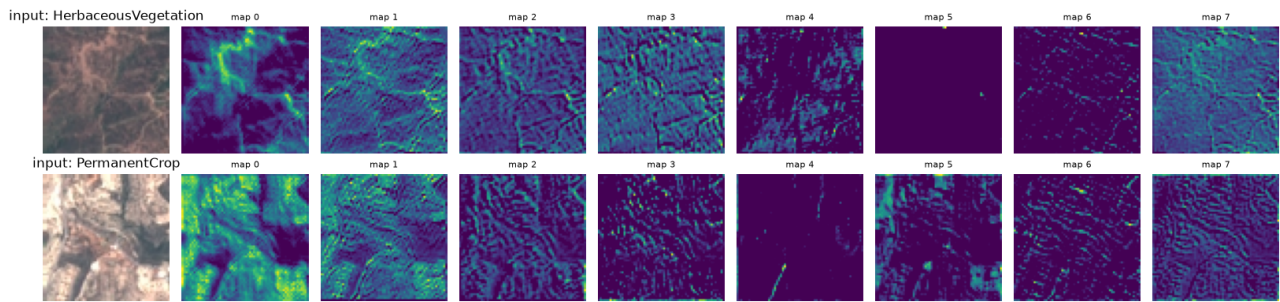


Figure 8. First 8 activation maps of Conv1 for two test images.

The filters act as edge and colour detectors. In the activation maps some channels respond to **edges and boundaries** (field borders, paths), others to **bright or uniform colour regions**, others to fine **texture**. Early layers capture low-level information; deeper layers combine it into larger patterns.

5.3 Discussion: complexity vs cost vs performance

- **More parameters is not better:** the MLP has 3× more parameters than the CNN but is 29 points worse. Convolution uses local connections and weight sharing.
- **The CNN is slower per epoch than the MLP** although it is smaller, because each filter slides over every image position (more computation).
- **ResNet18** is the largest and slowest per epoch, but needs only 3 epochs, has the lowest total training time and the best accuracy (+6.8 points over the CNN). Its cost is a larger model and slower inference.

6. Conclusion

Summary. CNNs clearly beat MLPs on images (90.3% vs 61.6%) with fewer parameters. Transfer learning with ResNet18 gave the best accuracy (97.1%) in only 3 epochs. Adam converged fastest among the optimizers, and dropout reduced overfitting.

Limitations. Short training and no hyperparameter search; a single seed and split; only the RGB bands are used (EuroSAT has 13 spectral bands); images only from Europe.

Possible improvements. Longer training with a learning-rate schedule and early stopping; Batch Normalization in the CNN; stronger augmentation (rotations, colour jitter); larger input sizes or other pretrained models (ResNet50, EfficientNet); using all spectral bands; averaging results over several seeds.

Reproducibility. All code, plots and results are in *midterm.ipynb*. The dataset is downloaded automatically, all random seeds are fixed, and running the notebook twice produced identical accuracies.